DEEMED TO BE UNIVERSITY

Meta Compiler Framework: A Configurable Compiler Generator for Designing and Implementing Programming Languages

Compiler Design Project Review-1

Date& Time

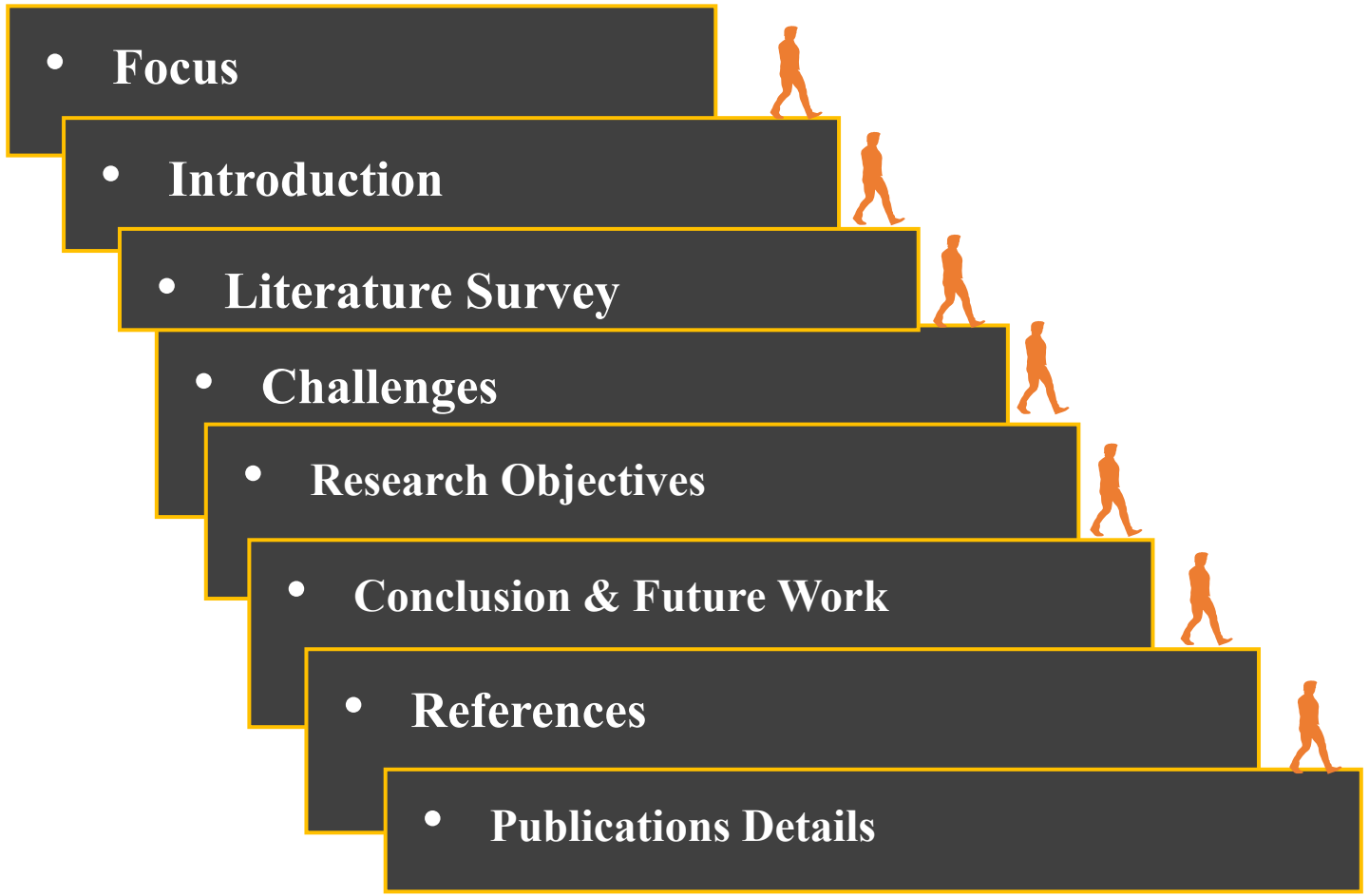
Under the Guidance of

Dr. V Anu Raghavi
Assistant Professor,
Department of Computer Science and Engineering,
KLH CSE Bowrampet Campus

Team Members

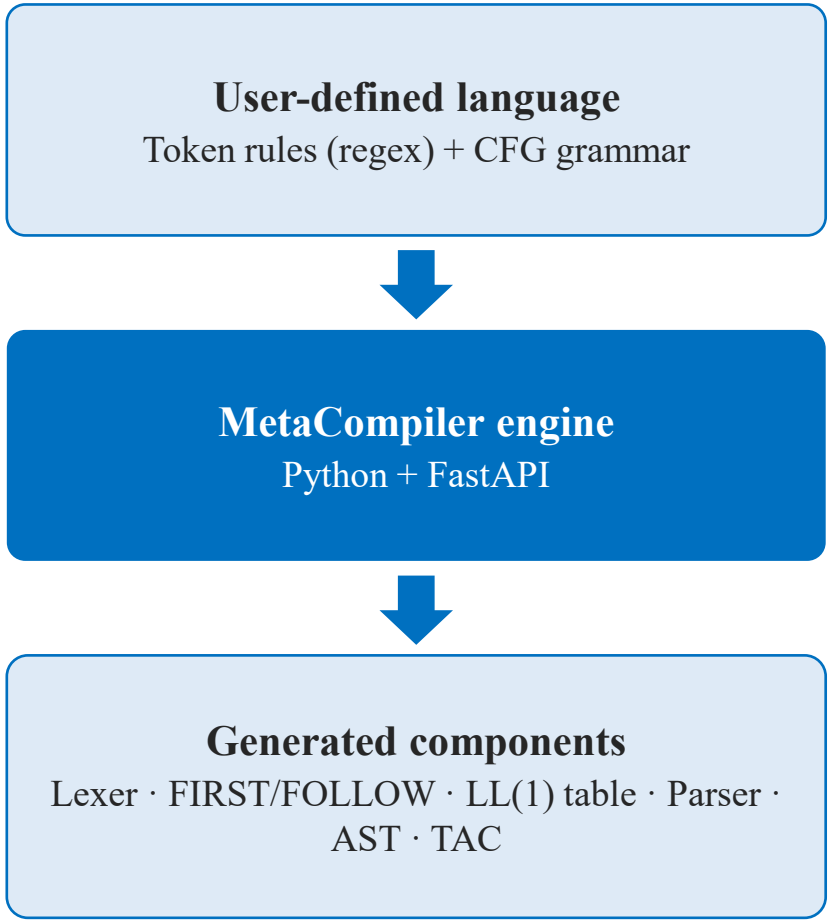
Name: Mithileshwar
Roll Number: 2420090151
Name: Rohith Reddy
Roll Number:2420030138
Name:Durga Reddy
Roll Number:2420030281

Outline of Project Work



Problem Statement

- Building a compiler by hand means writing separate lexical, syntax, semantic, intermediate-code and optimization modules.
- Doing this for every new language is time-consuming and demands deep programming knowledge.
- Students rarely get to see how source code is transformed inside each compiler phase.
- Need: a configurable platform where a language is defined by token rules (regex) and grammar productions, and the compiler components are produced automatically and demonstrated end to end.



DEEMED TO BE UNIVERSITY

Objectives

1. **Define languages**
   Token rules (regex), keywords, operators and an LL(1) grammar, with MiniLang as the sample.

2. **Lexical analysis**
   Tokenize source with line and column tracking and meaningful lexical errors.

3. **Grammar analysis**
   Compute FIRST/FOLLOW sets, build the LL(1) table and report conflicts.

4. **Syntax analysis**
   Table-driven predictive parser with step-by-step stack trace, parse tree and AST.

5. **Semantics and code**
   Symbol table, type checks, three-address code and basic optimizations.

6. **Visual studio**
   React + FastAPI web app that shows every compiler phase and the full pipeline.

Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation/Future work
Lesk & Schmidt [1]; Johnson [2]	Lex: A Lexical Analyzer Generator; Yacc: Yet Another Compiler-Compiler (Bell Labs, 1975)	Regular expressions compiled to a scanner (Lex); LALR(1) parser generation (Yacc)	Showed that a scanner and parser can be generated from a specification instead of coded by hand	Output is C code that must be built separately; no view of intermediate steps
Parr & Quong [3]	ANTLR: A Predicated-LL(k) Parser Generator, Software: Practice and Experience, 1995	Top-down LL(k) parsing extended with semantic and syntactic predicates	Top-down parsers generated from grammars are readable and practical to use	Generates code for a target language; not built to show parsing tables or steps to learners
Kats & Visser [4]	The Spoofax Language Workbench, OOPSLA 2010	Declarative specification of syntax, names, types and IDE support for a language	A language definition can drive a complete development environment	Large toolchain and steep learning curve for beginners

Problem Gap Identification

Regular Compiler		Meta Compiler Framework
User writes source programs.	→	User defines a programming language.
Compiler processes the program.	→	Framework generates compiler components.
Uses an existing language.	→	Can support newly defined languages.
Produces executable or target code.	→	Produces a compiler or compiler components.

Innovation, Creativity, and Novelty

Specification-driven

A language is defined by regex token rules and CFG productions; the compiler components are derived from it instead of being coded by hand.

Real algorithms, no hardcoded output

FIRST/FOLLOW with epsilon handling, an LL(1) table with conflict detection, and a table-driven predictive parser.

Every phase made visible

Tokens, parsing stack steps, parse tree, AST, symbol table, TAC and before/after optimization can all be inspected.

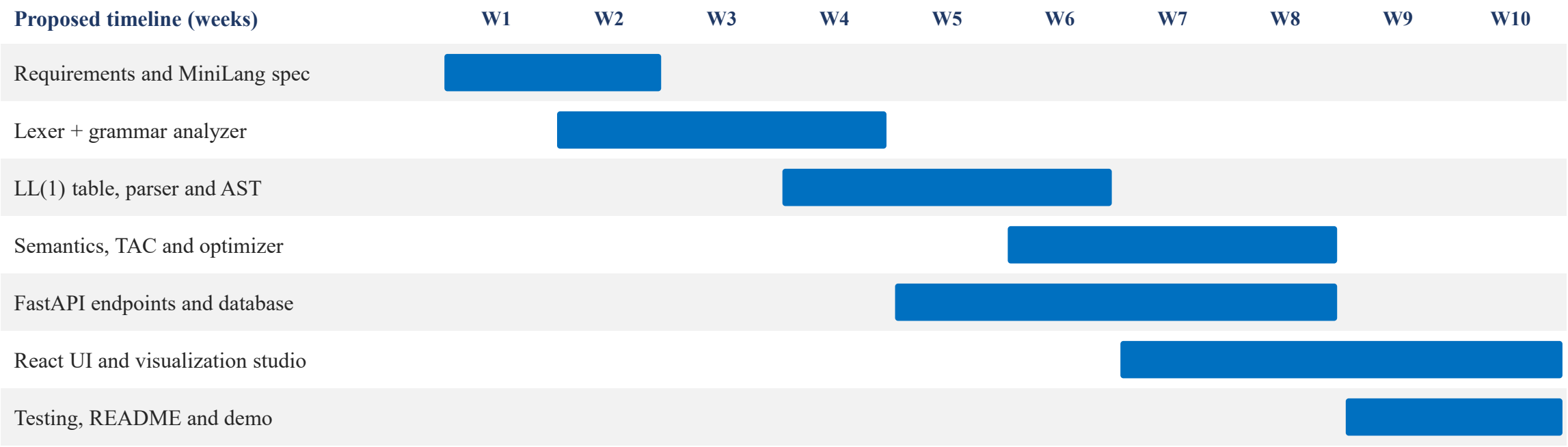
Interactive compiler studio

A React Flow pipeline shows status, time, output and errors for each stage, alongside a Monaco code playground.

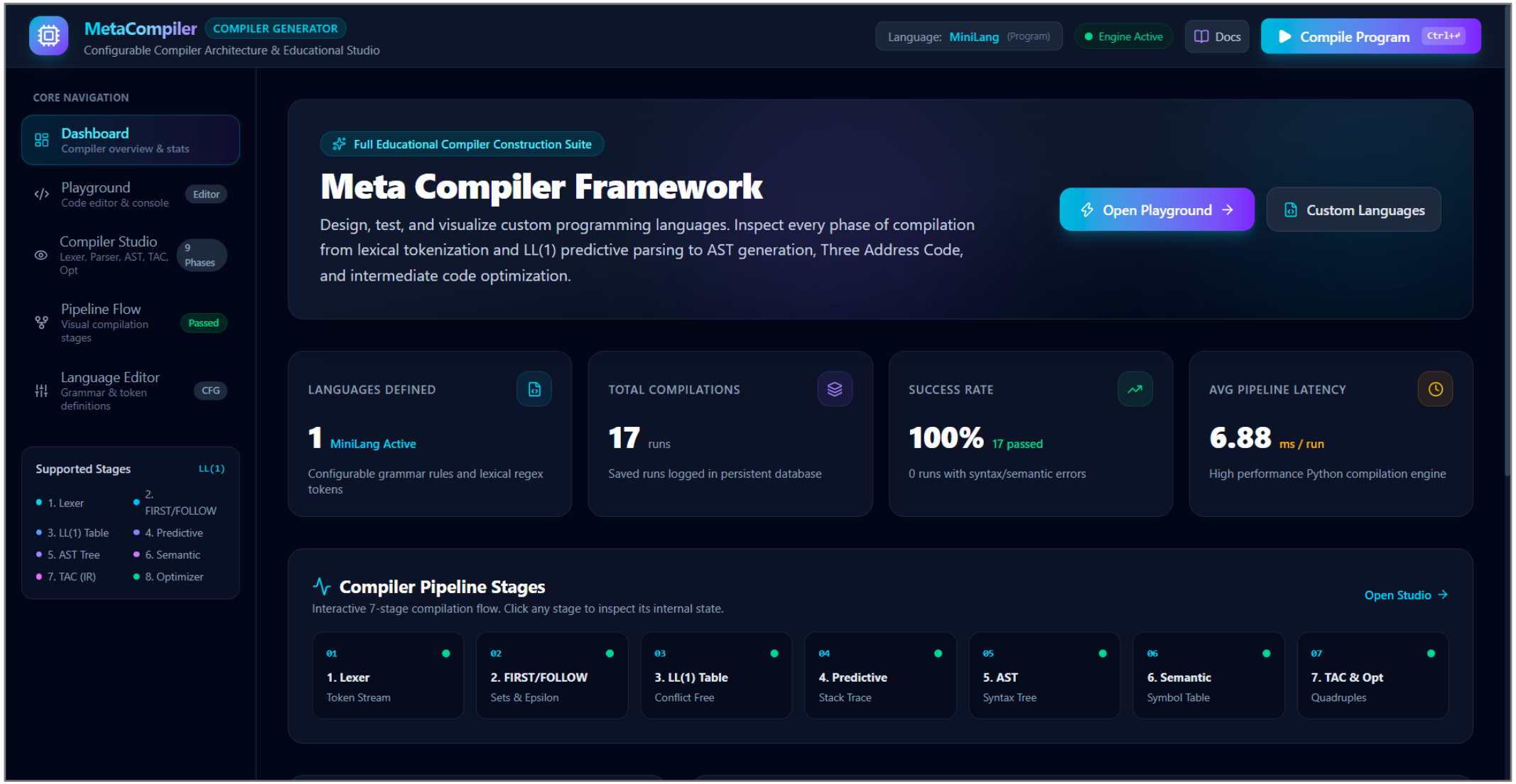
Scope of the initial version: LL(1) grammars and the MiniLang subset; support for arbitrary programming languages is not claimed.

Feasibility and Project Plan

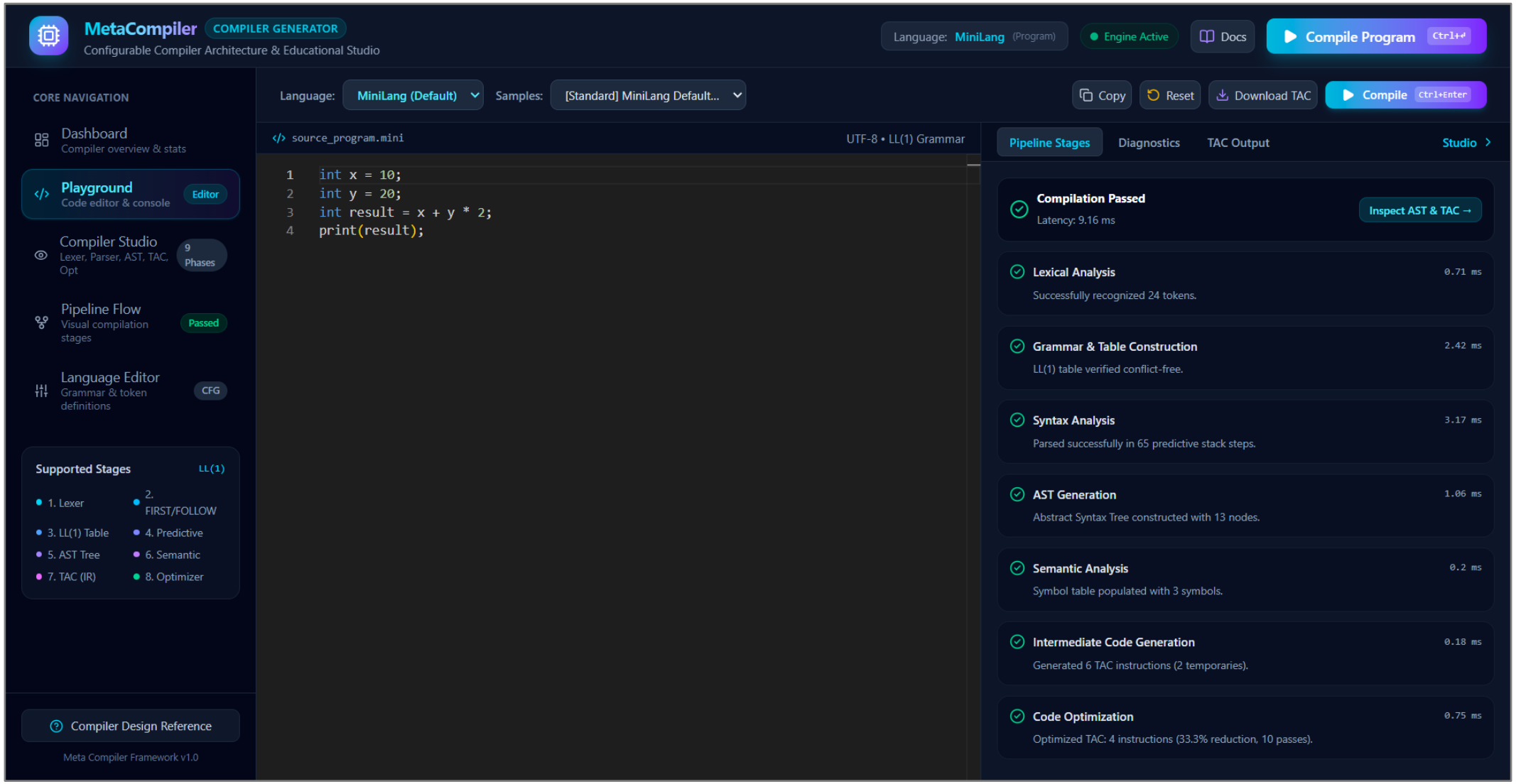
- Technically feasible: a Python/FastAPI backend, React + TypeScript frontend and SQLite (PostgreSQL-compatible) use mature, free tools.
- The algorithms (regex scanning, FIRST/FOLLOW, LL(1) table, predictive parsing, three-address code) are standard Compiler Design techniques.
- Scope is controlled: LL(1) grammars and the MiniLang subset, with the compiler engine built first and UI animations last.



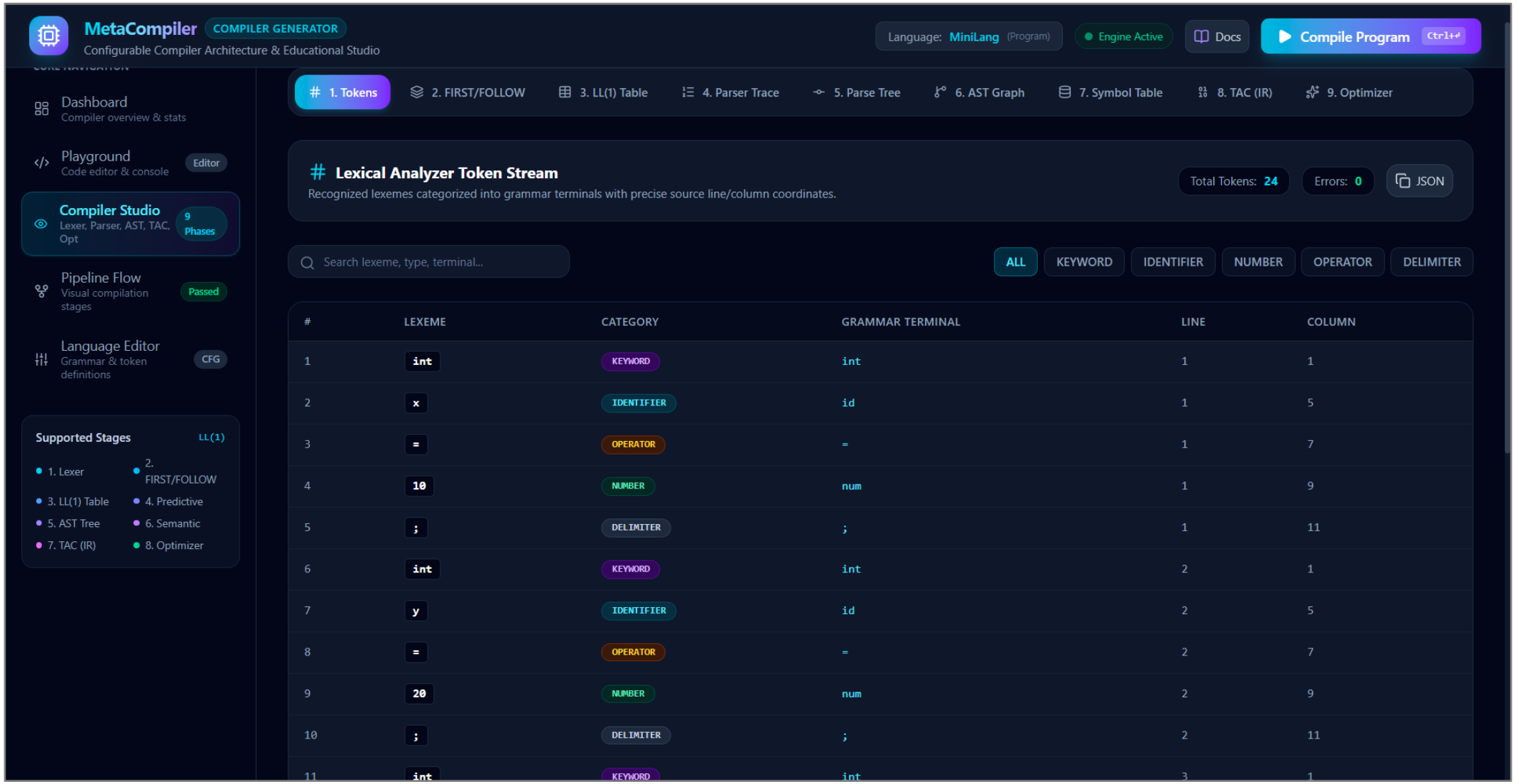
Output: Dashboard



Output: Playground and Compilation

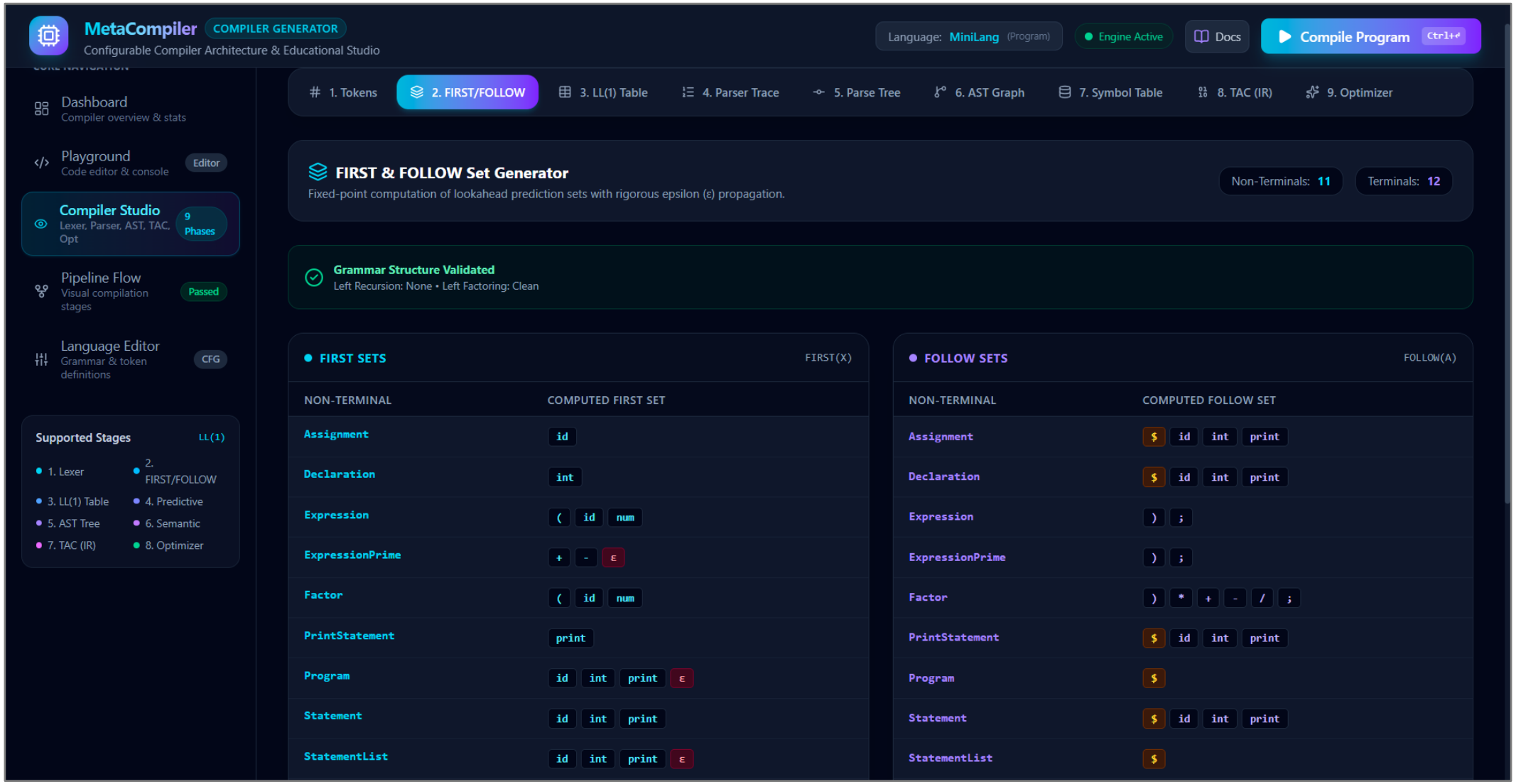


Output: Lexical Analysis (Tokens)



#	LEXEME	CATEGORY	GRAMMAR TERMINAL	LINE	COLUMN
1	int	KEYWORD	int	1	1
2	x	IDENTIFIER	id	1	5
3	=	OPERATOR	=	1	7
4	10	NUMBER	num	1	9
5	;	DELIMITER	;	1	11
6	int	KEYWORD	int	2	1
7	y	IDENTIFIER	id	2	5
8	=	OPERATOR	=	2	7
9	20	NUMBER	num	2	9
10	;	DELIMITER	;	2	11
11	int	KEYWORD	int	3	1

Output: FIRST and FOLLOW Sets



DEEMED TO BE UNIVERSITY

Output: Code Optimization (TAC)

